Writing an adapter

Developer quick-sheet · for anyone who wants goldpan to read another file format · version 1, 26 October 2026

An **adapter** is the only code that knows a file format. It turns one storage (a MIDAS file, an idx dump, a ROOT file, your lab's own format) into the objects the rest of `goldpan` works on; everything else in the library sees arrays and never a file. Four exist: `idx`, `parquet`, `midas` (they read traces into an `Events`) and `rq_root` (it reads processed quantities into a table). A new one is a small Python module. Start from the complete example in `tests/example_adapter.py` (in the goldpan repository) (about 50 lines, tested), then follow this sheet.

1. Which shape

- **Events adapter**: file to `goldpan.data.Events`: the traces of one detector (read-only arrays, one per channel) and one record of quantities per event. Use it for anything with waveforms. Examples: `idx`, `midas`, `parquet`.
- **Table adapter**: file to an Awkward Array with one record per event whose first fields are the key `series`, `dump`, `event`, then the quantities. Use it for processed values without traces. `goldpan.adapters.rq.attach(events, table)` joins it to an `Events`. Example: `rq_root`, the model for a new one: take the detector number from the user, fill a branch the file lacks with NaN instead of failing, and test it by joining it to an `Events` with `attach`.

2. The rules every adapter follows

Rule	Why
The user names the detector, the channels and the readout. `read(path, *, detector_name, config, ...)` with a `PulseConfig`; a channel map such as `{"PAS1": 0}` when the file numbers its channels. Never a default.	A wrong silent default (1.6 us for 0.8 us samples) changes every time and energy without an error.
Check what the file knows against what the user said: the sample period, the number of samples. Raise `ValueError` that says both numbers.	`midas` refuses a Run 76 readout on a Z7 file.
The key is `series`, `dump`, `event`. Event numbers restart in every dump. Take series and dump from the file name unless the user gives them.	Without the dump, two events share a number.
Build with `Events.from_arrays(...)` (or `Events(...)`). The constructor checks the shapes and the key, and makes the traces read-only.	You cannot return a malformed `Events`.
Keep the file's dtype (`uint16` stays `uint16`); times in seconds; `meta` holds provenance (`adapter`, `source`, run information) and only JSON-friendly values.	Parquet stores `meta` as JSON; no hidden conversions.
Skip and count, never invent. Events without the channels asked for are skipped and counted in `meta`. A file that stops in the middle ends without an error and says `complete: False`. Something you do not read yet (hybrid sampling) raises `NotImplementedError`.	A silent wrong join corrupts a result.
Read only what is asked for. Take a `channels` argument and a way to stop early (`max_events`); a raw file can be 15 GB.	`midas` reads past channels it was not asked for.

3. The functions

```
def read(path, *, detector_name, config, ...) -> Events          # required
def iter_chunks(path, *, chunk=256, **kw) -> Iterator[Events]    # optional
def write(events, path) -> None                                  # optional
```

`iter_chunks` gives the same events in pieces (for a file larger than memory); `write` makes a file that `read` reads back.

The core of `tests/example_adapter.py` (in the goldpan repository): it checks the sample period of the file, then

builds the `Events`.

```
with np.load(path) as f:
    names = list(channels) if channels else [k for k in f.files if k not in NOT_CHANNELS]
    if "period_s" in f.files and config.sample_period_s is not None:
        in_file = float(f["period_s"])
        if abs(in_file - config.sample_period_s) > 0.01 * config.sample_period_s:
            raise ValueError(f"the file samples every {in_file * 1e6:g} us but the readout "
                             f"you gave says {config.sample_period_s * 1e6:g} us")
    traces = {n: f[n] for n in names}
    event = f["event"] if "event" in f.files else None
n_samples = next(iter(traces.values())).shape[1]
detector = Detector(detector_name, tuple(ChannelInfo(n, role="phonon") for n in names),
                    Readout.single(n_samples, config))
return Events.from_arrays(detector, traces, series=series, dump=dump, event=event,
                          meta={"adapter": "npz", "source": str(path)})
```

4. Test it

Pass your module and a small sample file to the conformance test. It checks that `read` returns an `Events` that satisfies its rules, that `iter_chunks` joined equals `read`, and that `write` then `read` loses nothing (the same keyword arguments are used for every call); it returns the list of checks passed:

```
from goldpan.testing import check_adapter
check_adapter(my_module, "sample.npz", detector_name="TOY", config=CONFIG)
```

Then add the tests only you can write: the error messages, a truncated file, a channel that is missing. For data to try the adapter on, `tests/synthetic_series.py` writes one made-up series in idx, MIDAS, RQ Parquet, RQ ROOT and native Parquet (`python tests/synthetic_series.py OUTDIR`); add a writer for your format there and compare it with the same events. A test on a real file belongs in the repository only as a skipped-unless-present test: put the path in an environment variable, as `GOLDPAN_MIDAS_SAMPLE` does; real data are never committed.

5. Make it available

- **For yourself or your package:** `goldpan.adapters.register("name", my_module)` makes `goldpan.adapters.read(path, format="name", ...)` work. Nothing in `goldpan` changes.
- **As a built-in** (a pull request into `develop` of the goldpan repository, as in its `CONTRIBUTING.md`): the module `<name>.py` goes in `src/goldpan/adapters/` with a docstring that says what the format is and gives one example call (it becomes a page of the reference in the goldpan documentation); its public functions carry `@status(DONE)`; add it to `_BUILTIN` in `src/goldpan/adapters/__init__.py` (and to the suffix guess, if it has one); tests in `tests/` (`test_goldpan_<name>.py`); a line in its `CHANGELOG.md`.
- **A new dependency** is imported inside the functions that need it, with a message that says what to install (`ImportError: ... pip install lz4`), named as an extra in `pyproject.toml` of goldpan and pinned in `requirements.txt` of NSDF-Data (through the `goldpan[...]` line). It also needs a *Students, please:* line at the next NSDF-Data release (`RELEASING.md`).

6. What the four adapters learned

- **Look at a real file early.** `midas` was first written from the library's source, and a real file from S3DF then showed the sections the source did not make obvious: an extra block to read past, events of other kinds, and an end-of-run event that says whether the file is complete.
- **The file's own numbers disagree with your memory.** Event numbers, serial numbers and masks are not what the names suggest (the serial of the begin- and end-of-run events is the run number). Check and write it down in the docstring.
- **Do not guess a missing format feature.** Raise a clear error naming the feature, and add it when a file needs it.
- **Truncated and compressed files are normal.** Test a file cut in half; read `.gz` and `.lz4` as streams.
- **Keep `iter_chunks` a pass over the file**, not one read per chunk; say in the docstring what is known only at the end (`complete`).